L1 L2 ANDROID MASWE-0069 TEST

MASTG-TEST-0250: References to Content Provider Access in WebViews

Overview

This test checks for references to Content Provider access in WebViews which is enabled by default and can be disabled using the `setAllowContentAccess` method in the `WebSettings` class. If improperly configured, this can introduce security risks such as unauthorized file access and data exfiltration.

The JavaScript code would have access to any content providers on the device such as:

- declared by the app, **even if they are not exported**.
- declared by other apps, **only if they are exported** and if they are not following recommended best practices ↗ to restrict access.

Refer to *WebViews* for more information on the `setAllowContentAccess` method, the specific files that can be accessed and the conditions under which they can be accessed.

Example Attack Scenario:

Suppose a banking app uses a WebView to display dynamic content. The developers have not explicitly set the `setAllowContentAccess` method, so it defaults to `true`. Additionally, JavaScript is enabled in the WebView as well as the `setAllowUniversalAccessFromFileURLs` method.

1. An attacker exploits a vulnerability (such as an XSS flaw) to inject malicious JavaScript into the WebView. This could occur through a compromised or malicious link that the WebView loads without proper validation.
2. Thanks to `setAllowUniversalAccessFromFileURLs(true)`, the malicious JavaScript can issue requests to `content://` URIs to read locally stored files or data exposed by content providers. Even those content providers from the app that are not exported can be accessed because the malicious code is running in the same process and same origin as the trusted code.
3. The attacker-controlled script exfiltrates sensitive data from the device to an external server.

Note 1: We do not consider `minSdkVersion` since `setAllowContentAccess` defaults to `true` regardless of the Android version.

Note 2: The provider's `android:grantUriPermissions` attribute is irrelevant in this scenario as it does not affect the app itself accessing its own content providers. It allows **other apps** to temporary access URIs from the provider even though restrictions such as `permission` attributes, or `android:exported="false"` are set. Also, if the app uses a `FileProvider`, the `android:grantUriPermissions` attribute must be set to `true` by definition ↗ (otherwise you'll get a `SecurityException: Provider must grant uri permissions"`).

Note 3: `allowUniversalAccessFromFileURLs` is critical in the attack since it relaxes the default restrictions, allowing pages loaded from `file://` to access content from any origin, including `content://` URIs.

If this setting is not enabled, the following error will appear in `logcat`:

```
[INFO:CONSOLE(0)] "Access to XMLHttpRequest at 'content://org.owasp.mastestapp.provider/sensitive.txt'
from origin 'null' has been blocked by CORS policy: Cross origin requests are only supported
for protocol schemes: http, data, chrome, https, chrome-untrusted.", source: file:/// (0)
```

While the `fetch` request to the external server would still work, retrieving the file content via `content://` would fail.

Steps

1. Use a tool like semgrep to search for references to:
   - the `WebView` class.
   - the `WebSettings` class.
   - the `setJavaScriptEnabled` method.
   - the `setAllowContentAccess` method from the `WebSettings` class.
   - the `setAllowUniversalAccessFromFileURLs` method from the `WebSettings` class.
2. Obtain all content providers declared in the app's AndroidManifest.xml file.

Observation

The output should contain:

- A list of WebView instances including the following methods and their arguments:
  - `setAllowContentAccess`
  - `setJavaScriptEnabled`
  - `setAllowUniversalAccessFromFileURLs`
- A list of content providers declared in the app's AndroidManifest.xml file.

Evaluation

Fail:

The test fails if all of the following are true:

- `setJavaScriptEnabled` is explicitly set to `true`.
- `setAllowContentAccess` is explicitly set to `true` or *not used at all* (inheriting the default value, `true`).
- `setAllowUniversalAccessFromFileURLs` method is explicitly set to `true`.

You should use the list of content providers obtained in the observation step to verify if they handle sensitive data.

Note: The `setAllowContentAccess` method being set to `true` does not represent a security vulnerability by itself, but it can be used in combination with other vulnerabilities to escalate the impact of an attack. Therefore, it is recommended to explicitly set it to `false` if the app does not need to access content providers.

Pass:

The test passes if any of the following are true:

- `setJavaScriptEnabled` is explicitly set to `false` or *not used at all* (inheriting the default value, `false`).
- `setAllowContentAccess` method is explicitly set to `false`.
- `setAllowUniversalAccessFromFileURLs` method is explicitly set to `false`.

Best Practices

MASTG-BEST-0011: Securely Load File Content in a WebView

MASTG-BEST-0012: Disable JavaScript in WebViews

MASTG-BEST-0013: Disable Content Provider Access in WebViews

Demos

MASTG-DEMO-0029: Uses of WebViews Allowing Content Access with semgrep